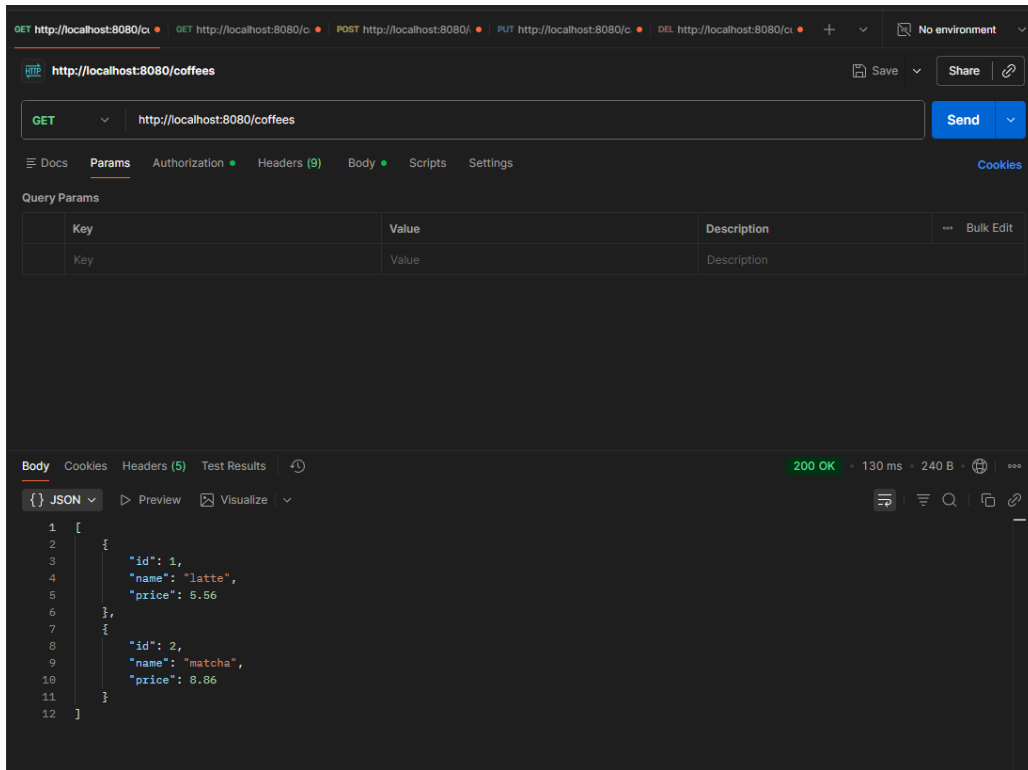


## 1. Get all coffee



GET http://localhost:8080/coffees

Send

Query Params

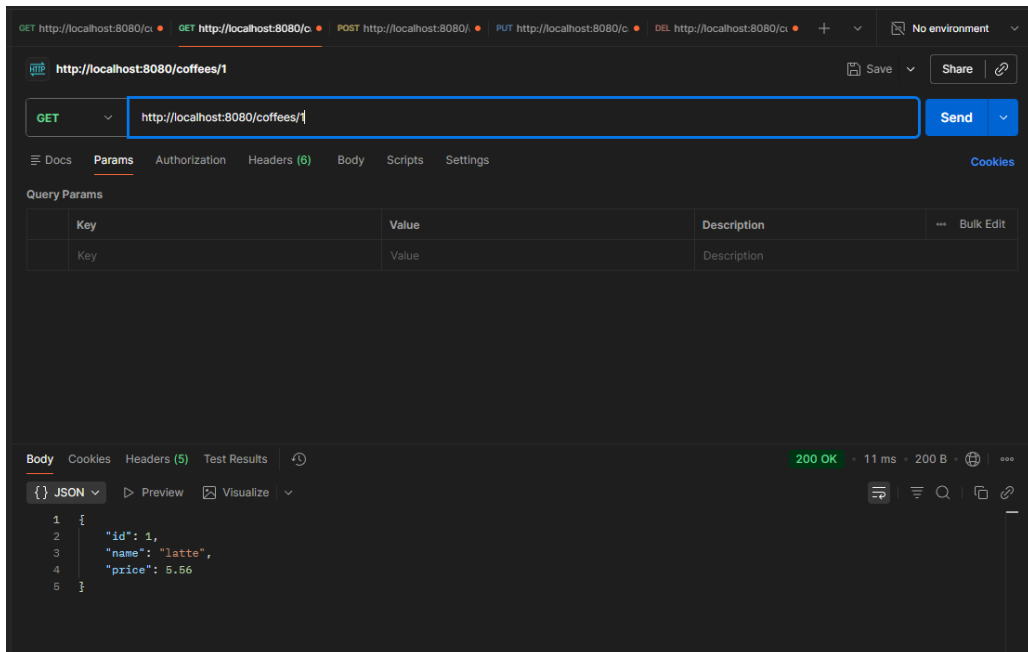
| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

Body

200 OK · 130 ms · 240 B

```
1 [
2   {
3     "id": 1,
4     "name": "latte",
5     "price": 5.56
6   },
7   {
8     "id": 2,
9     "name": "matcha",
10    "price": 8.86
11  }
12 ]
```

## 2. Get by id



GET http://localhost:8080/coffees/1

Send

Query Params

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

Body

200 OK · 11 ms · 200 B

```
1 {
2   "id": 1,
3   "name": "latte",
4   "price": 5.56
5 }
```

### 3. Get by id (not found)

The screenshot shows a REST client interface with a dark theme. At the top, a list of requests is visible, with the current one being a GET request to `http://localhost:8080/coffees/999`. The main area shows the request details for this endpoint, with the method set to GET. Below the request bar, there are tabs for Docs, Params, Authorization, Headers (6), Body, Scripts, and Settings. The Params tab is active, showing a table for Query Params with columns Key, Value, and Description. The table is currently empty. At the bottom, the Body tab is selected, displaying a 404 Not Found error status. The response details show a status of 404 Not Found, a response time of 4 ms, and a body size of 130 B. The response body is shown in raw format.

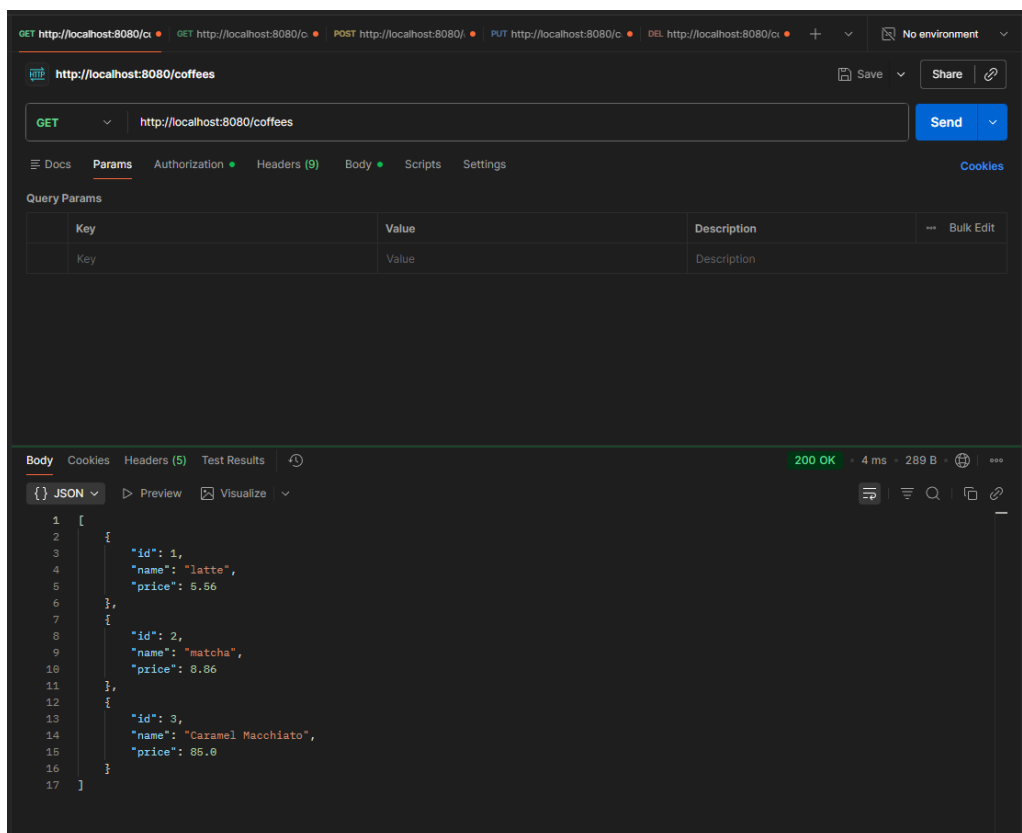
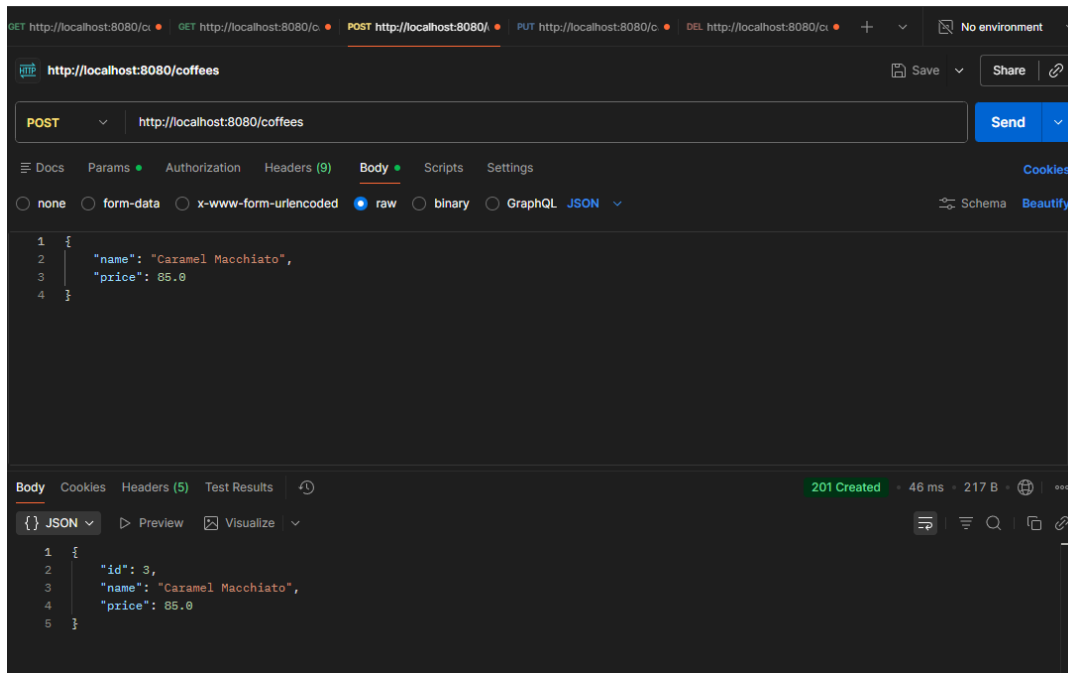
GET `http://localhost:8080/coffees/999`

Query Params

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

Body: 404 Not Found • 4 ms • 130 B

#### 4. Post coffee



## 5.put

The screenshot shows a REST client interface with a PUT request to `http://localhost:8080/caffe/1`. The request body is a JSON object: `{ "price": 100.0 }`. The response is `200 OK` with a status of `6 ms` and a body size of `201 B`. The response body is a JSON object: `{ "id": 1, "name": "latte", "price": 100.0 }`.

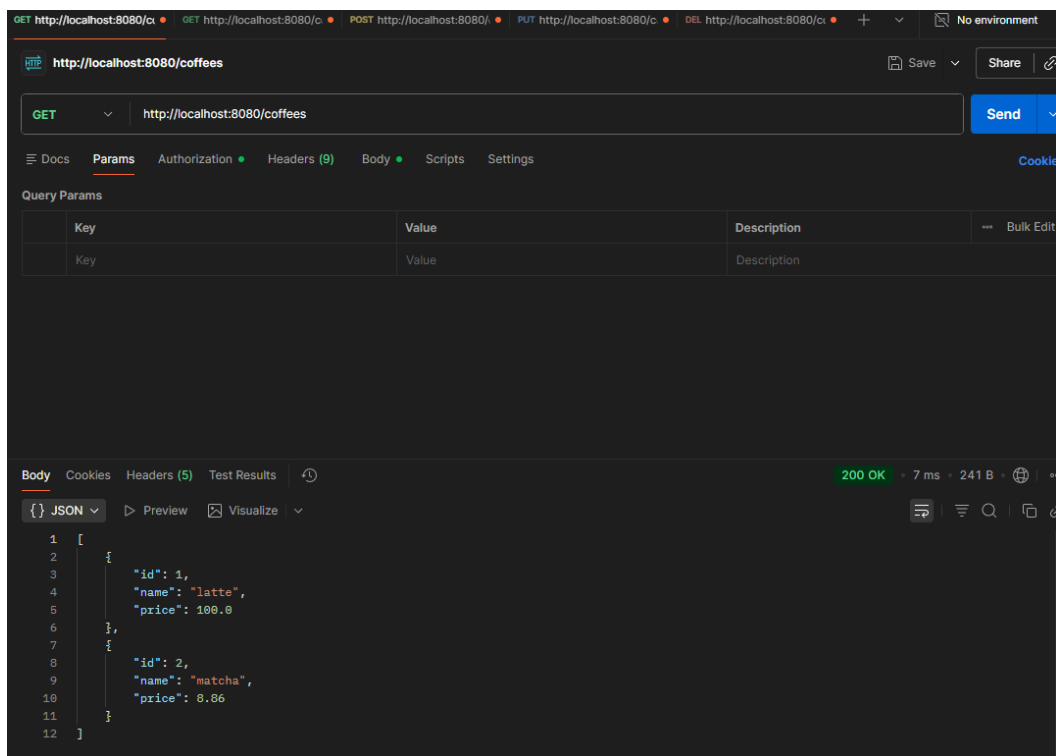
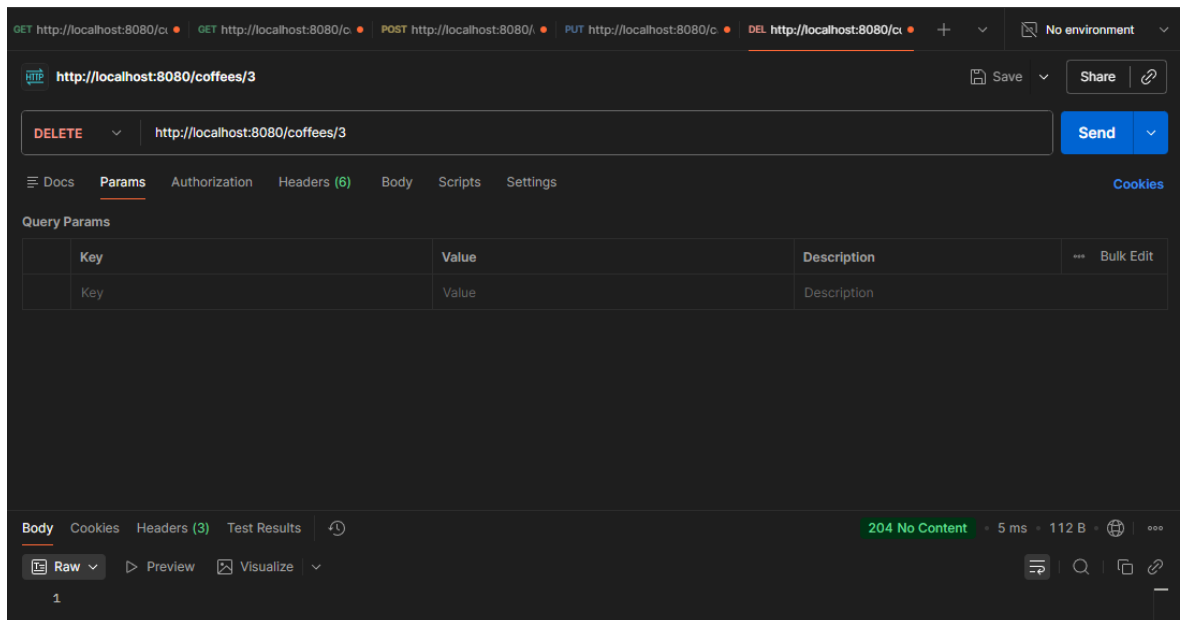
```
1 {
2   "price": 100.0
3 }
```

```
1 {
2   "id": 1,
3   "name": "latte",
4   "price": 100.0
5 }
```

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/caffe`. The response is `200 OK` with a status of `4 ms` and a body size of `290 B`. The response body is a JSON array of three objects: `[ { "id": 1, "name": "latte", "price": 100.0 }, { "id": 2, "name": "matcha", "price": 8.86 }, { "id": 3, "name": "Caramel Macchiato", "price": 85.0 } ]`.

```
1 [
2   {
3     "id": 1,
4     "name": "latte",
5     "price": 100.0
6   },
7   {
8     "id": 2,
9     "name": "matcha",
10    "price": 8.86
11  },
12  {
13    "id": 3,
14    "name": "Caramel Macchiato",
15    "price": 85.0
16  }
17 ]
```

## 6 Delete



End point แต่ละตัวทำงานได้ถูกต้อง

## ตอบคำถาม

### 1. ความแตกต่างของ HTTP Method (ตัวอย่างจากโปรเจกต์ร้านกาแฟ)

- GET ใช้ดึงข้อมูล
- POST ใช้ส่งข้อมูลเพื่อสร้างของใหม่
- PUT ใช้อัปเดตข้อมูลเดิมที่มีอยู่แล้ว
- DELETE ใช้ลบข้อมูล

### 2. ทำไมต้องแยก Controller กับ Service?

- เพราะทำให้โค้ดเป็นระเบียบ บำรุงรักษาง่ายขึ้น ตรวจสอบข้อผิดพลาดได้ตรงจุด และสามารถนำ Service ไปเรียกใช้ซ้ำได้ง่าย

### 3. ข้อมูลใน List หายไปตอนไหน และวิธีแก้

- ข้อมูลจะหายไปทันทีที่ปิดโปรแกรม เพราะถูกเก็บไว้ใน RAM
- ถ้าไม่อยากหายต้องเก็บข้อมูลลง Database หรือเขียนลงฮาร์ดดิสก์อย่าง

### 4. หน้าที่ของ Annotations แต่ละตัว

- @RestController: ดึงออก Spring Boot ว่าเป็นตัวจัดการ API และให้แปลงผลลัพธ์ กลับไปเป็น JSON แบบอัตโนมัติ
- @GetMapping: ใช้ผูก URL กับคำสั่ง GET เพื่อรับการเรียกดูข้อมูล
- @PostMapping: ใช้ผูก URL กับคำสั่ง POST เพื่อรับการสร้างข้อมูลใหม่
- @PathVariable: ใช้ดึงตัวเลขหรือข้อความที่แฝงมากับ URL มาใช้งาน เช่น ดึงเลข 1 จาก /coffees/1
- @RequestBody: ใช้ดึง ก่อนข้อมูล JSON ที่ส่งมาใน Body มาแปลงเป็นออบเจกต์ในภาษา Java ทันที